

Implementation Note — Speech Understanding PA2

Mayur Shriram Gaidhane | M22AIE248 | IIT Jodhpur
github.com/Mayur-S-Gaidhane/speech_assignment2

Part I (Task 1.2) — Length normalization while selecting best Whisper candidate

In `ngram_lm.py`, the `score()` function returns a raw sum of bigram log-probabilities. When I first tested the candidate selection in `transcribe.py`, I was directly comparing these raw scores across the five Whisper beam outputs. The problem I ran into was that the LM was always picking shorter candidates. For example it was preferring a cut-off 10-word output over a complete 20-word transcription, even when the longer one was clearly more accurate. After looking at the scores printed during testing, I realized that a shorter sentence naturally gives a less negative log-probability sum just because fewer log terms are added — it has nothing to do with quality. So in the `select_best()` function in `transcribe.py` I added one line: `score = raw_score / max(len(t.split()), 1)`. After this fix the LM correctly ranked the best candidate with score -4.2220 at the top, which selected 'text encoder' over 'text in Coder' from the beam outputs. The other candidates scored -4.4425, -4.4575 and so on, which matched what I saw printed in the pipeline output.

Part II (Task 2.1) — Order of IPA mapping and g2p-en matters

I have two separate files for IPA conversion. The custom Hinglish mapping is in `phonetic/ipa_mapping.py` and the general English phoneme conversion is done by `g2p-en` inside `ipa_converter.py`. Initially I had written the code so that `g2p-en` ran first on the full transcript and then I applied my custom mapping on top. The output was wrong for Hindi words — for instance the word 'phir' was getting the English 'ph' pronunciation (like in 'phone') instead of the Hindi aspirated 'p' sound. The reason is that `g2p-en` sees 'ph' and immediately treats it as the English /f/ phoneme. By the time my mapping runs, the phoneme is already set wrongly. I fixed this by making sure `convert_to_ipa()` in `ipa_mapping.py` runs on the raw text first, replacing things like 'th' with 't^h' and 'ph' with 'p^h', before `g2p-en` ever sees the input. With this order the Hindi phonemes came out correctly in the ARPAbet output.

Part III (Task 3.2) — Stacking F0 and energy before DTW, not running two separate DTW calls

In `dtw_prosody.py` I extract both F0 using `librosa.yin()` and RMS energy using `librosa.feature.rms()`. My first attempt ran `fastdtw()` twice — once on F0 alone and once on energy alone. The DTW distances were very different for the two runs, which meant the alignment paths were not matching each other. When I listened to the warped output, pitch and loudness felt out of sync in places. I changed the code to do `np.vstack((f0, energy)).T` so both features are combined into one array before passing to `fastdtw()`. Now there is one single alignment path that applies to both F0 and energy at the same time, so they stay in sync. The final DTW distance was 1715.69 with an alignment path of 19177 frames. The MCD also dropped significantly from around 4.20 with flat gTTS to 0.19 after applying this warping.

Part IV (Task 4.1) — Using linear STFT instead of Mel spectrogram for LFCC features

In `lfcc_classifier.py` I compute the spectrogram using `np.abs(librosa.stft(y))` which gives a linear frequency spectrogram. Then I pass this to `librosa.feature.mfcc()` with the `S` parameter instead of computing it the normal way from raw audio. The reason I did it this way is that the normal `librosa.feature.mfcc()` call internally uses a Mel filterbank. For spoofing detection I wanted linear frequency spacing because gTTS output has artifacts at higher frequencies (roughly 4-8 kHz range) that the Mel scale tends to squash together. By providing the linear STFT spectrogram directly, I get coefficients that are more sensitive to those higher frequency differences between real and synthesized speech. The spoof score for my clean audio came out as 76.15, well above the threshold of 20, so it was correctly classified as Bona Fide. The EER came out at 8.0 percent which is under the required 10 percent limit.

All files mentioned above are in the GitHub repository. The full pipeline output showing all scores and metrics is in `results.txt`.